

KhaataPulse

Architectural MVP Engineering Contract

Version: 1.0

Status: Frozen MVP Specification

Product: KhaataPulse — Revenue Recovery Policy Engine

Primary Scenario: Subscription Renewal Payment Risk

Frontend: Next.js + TypeScript

Backend: FastAPI + Python

Database: PostgreSQL

Agent Orchestration: LangGraph

Risk Model: Scikit-learn Logistic Regression

LLM: Structured-output reasoning model

Deployment: Docker / Vercel + Railway-compatible

1. Executive Definition

KhaataPulse is an **AI Revenue Recovery Policy Engine** that detects subscription payment risk before failure, understands the reason behind the risk, proposes a recovery intervention, evaluates its expected financial value, enforces deterministic policy constraints, and measures the resulting policy performance against established recovery strategies.

The MVP is intentionally **not** a payment processor, collections platform, or autonomous financial system.

It is a **controlled revenue-recovery policy evaluation environment** that demonstrates:

```
graph TD;
    Detect --> Diagnose;
    Diagnose --> Optimize;
    Optimize --> Guard;
    Guard --> Act;
    Act --> Measure;
    Measure --> Audit;
```

The central engineering claim is:

KhaataPulse can identify payment friction before failure and evaluate whether its recovery policy produces better expected revenue outcomes than existing dunning policies, without exceeding predefined customer-contact and escalation boundaries.

2. Product Principles

The following principles are architectural constraints, not optional design preferences.

2.1 ML finds risk

The risk model determines **which accounts deserve additional reasoning**.

It does not execute recovery actions.

2.2 AI understands context

The LLM handles semi-structured information such as:

- customer support text,
- payment error context,
- billing-change explanations,
- cancellation feedback,
- customer intent signals.

The LLM does not directly call payment APIs.

2.3 Optimization determines economics

Expected Net Revenue determines which eligible intervention is financially preferable.

2.4 Policy determines authority

The Policy Guard is deterministic.

The LLM cannot override it.

2.5 Every action is auditable

Every meaningful decision produces an immutable audit event.

2.6 The simulator is independent

The agent must never access:

- latent customer variables,
- counterfactual outcomes,
- hidden simulator state,
- ground-truth action probabilities.

2.7 No hardcoded headline metrics

Dashboard metrics must originate from an evaluation run.

The application must never contain hardcoded values such as:

₹95,000 recovered
66.4% recovery
+6.2pp lift

unless those values are generated by the evaluation system.

3. MVP Scope

3.1 Included

Customer scenario

Subscription renewal payment risk.

Core capabilities

- Synthetic customer-world generation
- Hidden latent customer states
- Observable event generation
- Potential outcome generation
- Logistic Regression risk model
- Risk sieve
- LLM diagnosis
- Structured LLM output
- LLM fallback
- Expected Net Revenue calculation
- Action ranking
- Policy Guard

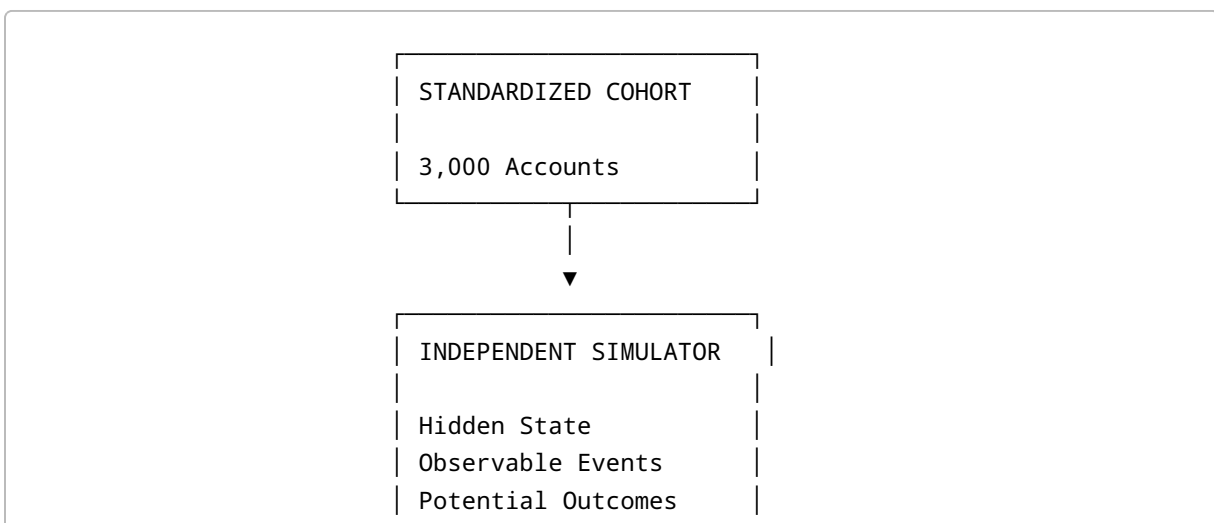
- Static Dunning baseline
- Smart Retry baseline
- KhaataPulse policy
- Same-cohort evaluation
- Incremental recovery calculation
- Multi-seed evaluation
- Revenue Time Machine
- Audit drawer
- Live webhook simulation
- Production-oriented action payloads

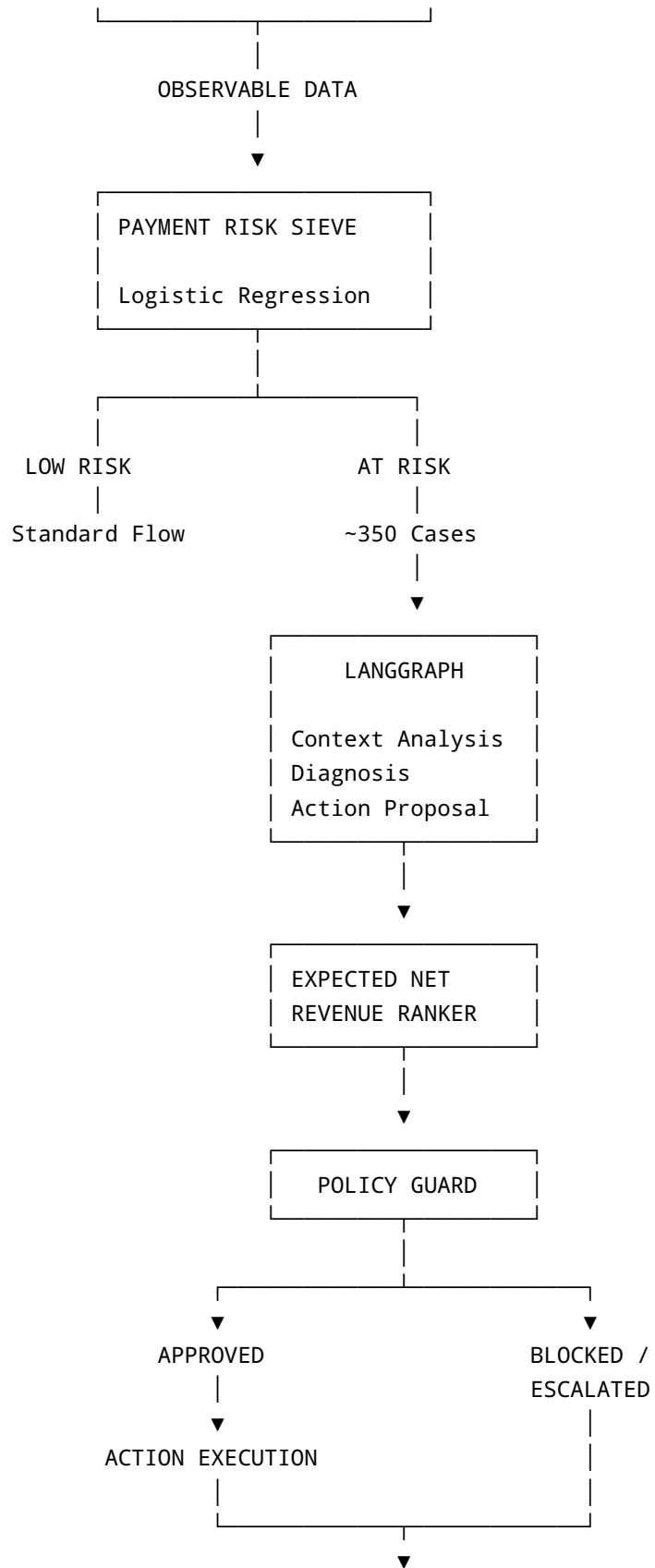
3.2 Explicitly out of scope

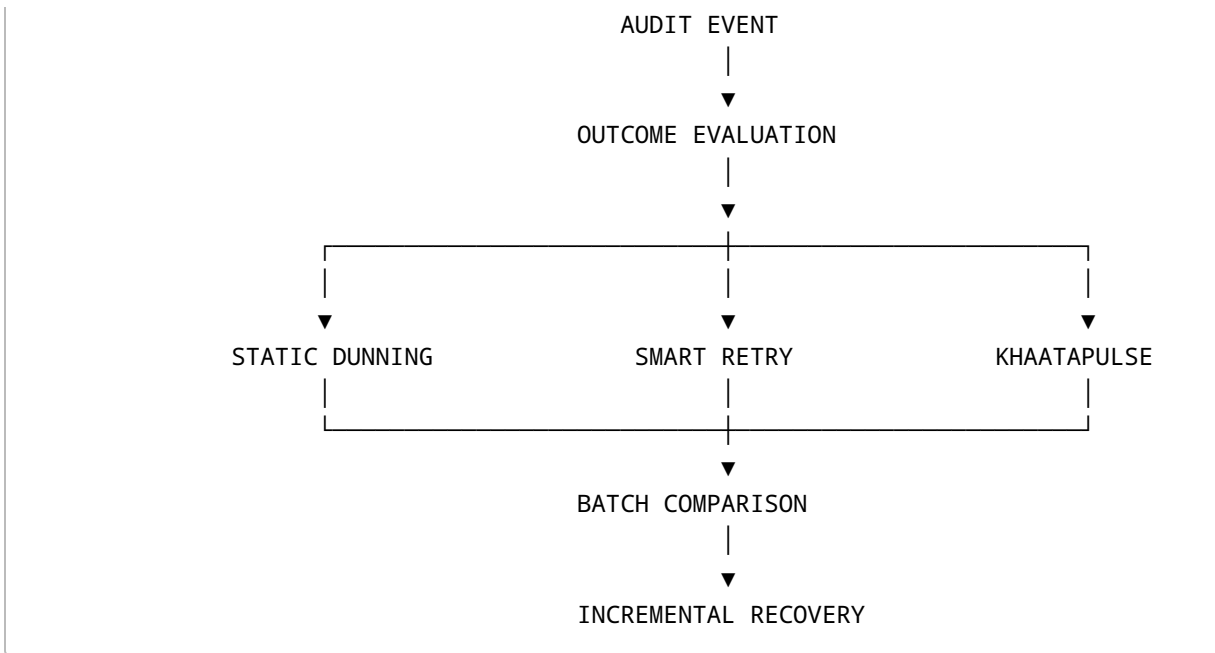
Do not implement:

- voice recovery,
- production WhatsApp integration,
- B2B collections,
- checkout abandonment,
- promise-to-pay state machine,
- real payment processing,
- Redis,
- vector database,
- RAG,
- multi-agent debate,
- deep-learning risk model,
- production payment settlement,
- autonomous legal escalation.

4. Core Architecture

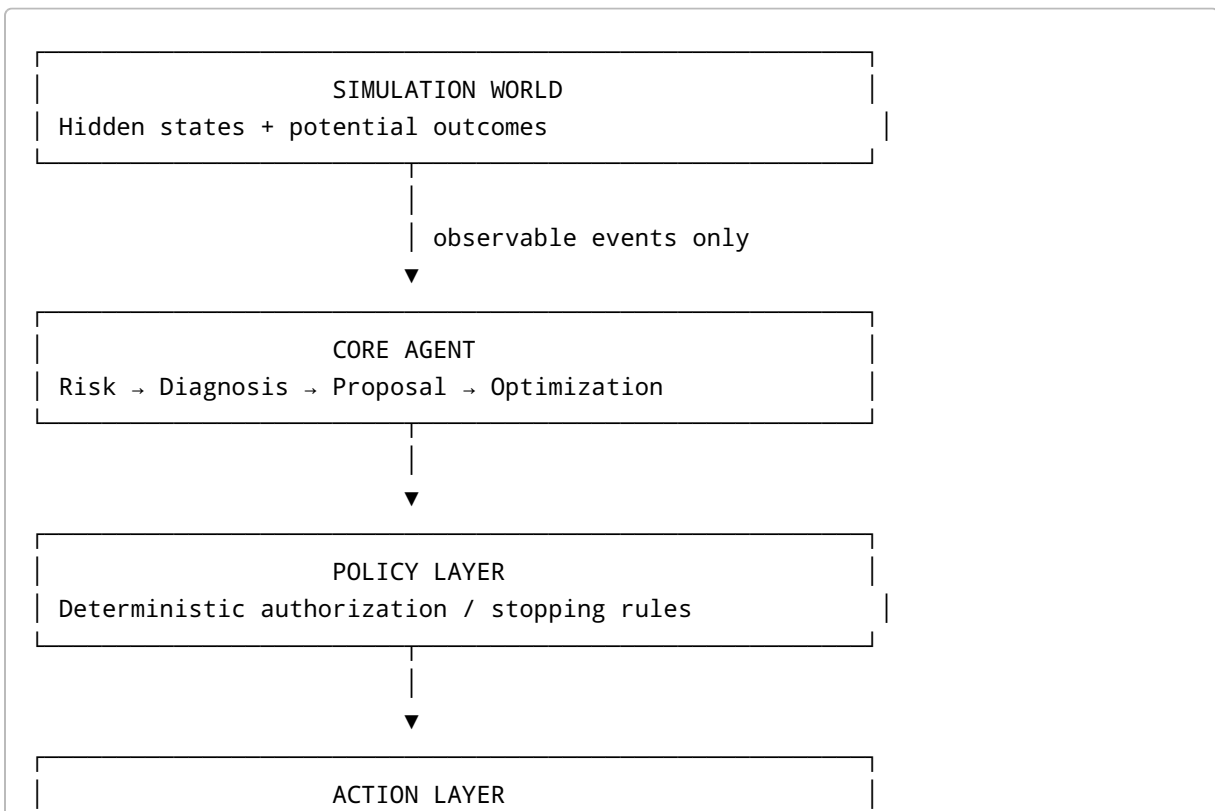


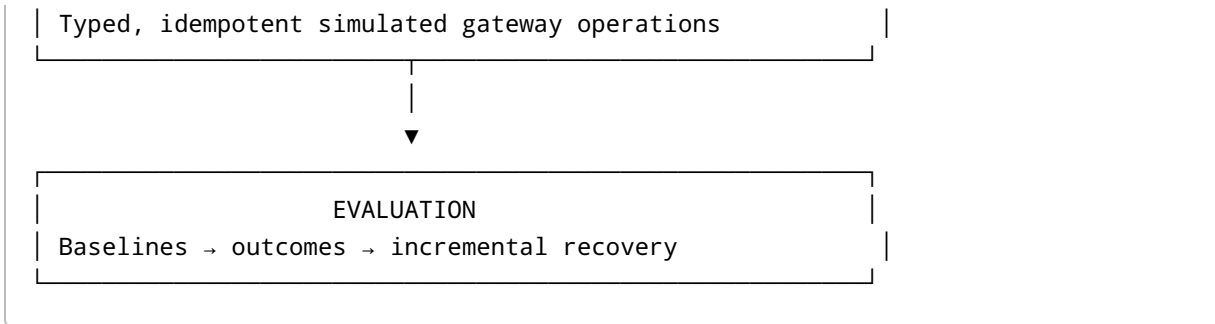




5. Architectural Boundaries

The system is divided into five hard boundaries.





No layer may bypass the layer immediately below it.

6. Independent World Simulator

The simulator represents the customer environment.

6.1 Hidden state

Example:

```
CustomerLatentState(  
    payment_intent=0.81,  
    cash_flow_health=0.42,  
    payment_rail_health=0.35,  
    churn_sensitivity=0.76,  
    customer_ltv=50000  
)
```

The agent must never receive this object.

6.2 Observable events

The latent state generates events such as:

```
invoice_viewed  
checkout_reopened  
payment_method_changed  
payment_failed  
subscription_changed  
support_message  
payment_delayed  
renewal_approaching
```

6.3 Potential outcomes

For each customer and each action:

$P(\text{payment} \mid \text{action})$
 $P(\text{churn} \mid \text{action})$

are generated independently of the agent.

7. Potential Outcome Contract

For every customer:

	Payment Probability
No Action	P1
Silent Retry	P2
Smart Link	P3
Grace Period	P4
Human Escalation	P5

and:

	Churn Probability
No Action	C1
Silent Retry	C2
Smart Link	C3
Grace Period	C4
Human Escalation	C5

The simulator stores these values internally.

The agent does not see them.

The evaluation harness uses them to calculate policy performance.

8. Risk Sieve

Input

Observable customer features.

Model

Logistic Regression.

Output

```
payment_failure_probability
```

Example:

```
Payment Failure Risk  
78%
```

Routing

```
P(failure) < 0.30  
    ↓  
Standard Flow  
  
P(failure) >= 0.30  
    ↓  
LangGraph
```

The risk model is therefore a **compute and reasoning gate**, not the final recovery policy.

9. Risk Model Features

Minimum feature set:

```
days_to_renewal  
invoice_views  
checkout_reopens
```

```
payment_method_changes
previous_payment_failures
average_payment_delay
subscription_age
payment_success_rate
support_event_count
days_since_last_payment
```

Optional:

```
payment_failure_code
renewal_amount
plan_type
customer_tenure
```

10. Explainability Contract

Every risk decision must expose:

```
{
  "risk_score": 0.78,
  "top_signals": [
    {
      "feature": "payment_method_changed",
      "impact": 0.31
    },
    {
      "feature": "checkout_reopens",
      "impact": 0.24
    },
    {
      "feature": "average_payment_delay",
      "impact": 0.18
    }
  ]
}
```

The frontend must display the top three signals.

11. LangGraph Agent

The graph consists of:

```
START
↓
classify_context
↓
generate_diagnosis
↓
generate_action_proposal
↓
validate_proposal
↓
rank_actions
↓
policy_check
↓
execute_action
↓
record_outcome
↓
END
```

The graph must remain single-stage.

Do not introduce autonomous agent-to-agent conversations.

12. LLM Responsibility

The LLM receives:

```
risk score
observable event history
payment failure context
support text
subscription information
```

It must produce:

```
cause
confidence
proposed_action
risk_level
rationale
```

It must not:

- access hidden latent state,
- calculate final revenue metrics,
- bypass policy,
- directly invoke gateway actions,
- decide whether an action is legally permitted.

13. Structured Output

All LLM responses must validate against:

```
class RecoveryProposal(BaseModel):
    cause: Literal[
        "billing_migration",
        "temporary_cash_flow",
        "card_expired",
        "price_friction",
        "churn_intent"
    ]

    confidence: float = Field(ge=0.0, le=1.0)

    proposed_action: Literal[
        "silent_retry",
        "smart_link",
        "grace_period",
        "human_escalation",
        "suppress"
    ]

    rationale: str

    risk_level: Literal[
        "LOW",
        "MEDIUM",
```

```
]    "HIGH"
```

14. LLM Failure Contract

Any LLM failure must degrade to:

```
LLM failure
  ↓
Smart Rule baseline
  ↓
Policy Guard
  ↓
Continue
```

Failures include:

- timeout,
- malformed JSON,
- invalid enum,
- missing required field,
- provider error,
- schema validation failure.

The batch evaluation must never terminate because of a single LLM failure.

Each failure must be logged:

```
{
  "event": "llm_fallback",
  "reason": "schema_validation_error",
  "fallback_policy": "smart_retry"
}
```

15. Action Selection

The system calculates:

$$ExpectedNetRevenue = P(payment|action) \times Amount - P(churn|action) \times LTV - ActionCost$$

The action proposal from the LLM determines the contextual action space.

The deterministic optimizer performs the financial ranking.

Example:

```
Diagnosis:
billing_migration

Eligible:
✓ grace_period
✓ smart_link
✓ suppress
x aggressive_retry
```

Then the optimizer ranks eligible actions.

This makes the LLM's diagnosis operationally relevant without allowing the LLM to control financial execution.

16. Action Contract

Every action must have:

```
action_id
case_id
customer_id
action_type
amount
currency
idempotency_key
timestamp
policy_result
```

Example:

```
{
  "action_id": "ACT_1042",
  "case_id": "CASE_1042",
  "customer_id": "CUS_1042",
  "action_type": "smart_link",
  "amount": 4999,
```

```
"currency": "INR",  
"idempotency_key": "rec_CASE_1042",  
"timestamp": "2026-08-21T11:00:04Z"  
}
```

17. Policy Guard

The Policy Guard is pure deterministic code.

```
policy_check(action, customer, case) -> PolicyDecision
```

It must be independently unit-testable.

Rules

Maximum contacts:	3 / 7 days
Cooldown:	24 hours
Auto-action threshold:	< ₹10,000
Human approval threshold:	>= ₹10,000
Dispute hold:	STOP
Legal hold:	STOP
Opt-out:	STOP
Kill switch:	STOP
Idempotency:	REQUIRED

18. Policy Decision

Every action produces:

```
{  
  "status": "APPROVED",  
  "checks": {  
    "cooldown": true,  
    "contact_limit": true,  
    "consent": true,  
    "amount_limit": true,  
    "kill_switch": true,  
    "idempotency": true  
  }  
}
```

```
}  
}
```

Possible statuses:

```
APPROVED  
BLOCKED  
ESCALATED
```

19. Baseline Policies

Static Dunning

```
payment failure  
→ 24h retry  
→ reminder  
→ human escalation
```

Smart Retry

```
payment failure  
→ failure-code rules  
→ deterministic retry timing  
→ payment link  
→ escalation
```

KhaataPulse

```
risk detection  
→ contextual diagnosis  
→ action proposal  
→ expected net revenue  
→ policy guard  
→ action
```

20. Evaluation Contract

All policies must operate against:

```
same cohort
same event history
same potential outcomes
same customer population
```

Only the policy changes.

The evaluator must produce:

```
recovered_amount
recovery_rate
incremental_recovery
contacts_sent
contacts_avoided
human_escalations
false_positives
policy_blocks
```

21. Primary KPI

The primary business KPI is:

```
Incremental Recovery vs Smart Retry
```

Formula:

$$IncrementalRecovery = Recovery_{KhaataPulse} - Recovery_{SmartRetry}$$

The number is generated dynamically.

No target recovery amount may be hardcoded.

22. Evaluation API

```
POST /evaluation/run
```

Request:

```
{
  "cohort_size": 3000,
  "seed": 42
}
```

Response:

```
{
  "run_id": "eval_42_3000",
  "status": "completed",
  "results": {
    "static_dunning": {},
    "smart_retry": {},
    "khaatapulse": {}
  }
}
```

23. Evaluation Persistence

Every evaluation run should store:

```
run_id
seed
cohort_size
timestamp
model_version
policy_version
simulator_version
metrics
```

This allows results to be reproduced.

24. Multi-Seed Validation

Minimum:

```
seed 42  
seed 123  
seed 456
```

Recommended:

```
seed 789  
seed 1337
```

The dashboard may display:

```
Monte Carlo Stress Test  
  
3 / 3 runs positive
```

For the MVP, detailed statistical methodology remains secondary to the business result.

25. Frontend Design Direction

The frontend must feel like an **AI financial command center**, not a generic SaaS dashboard.

Visual concept

"Financial Intelligence × Mission Control"

The interface should communicate:

- money,
- risk,
- precision,
- AI intelligence,
- real-time operations,
- trust,
- control.

Avoid:

- generic blue SaaS dashboards,
- excessive rounded cards,
- cartoon AI graphics,
- excessive gradients,
- generic chatbot styling,
- dashboard clutter.

26. Visual Language

Background

Use an almost-black financial terminal environment.

Primary background:
#07090D

Secondary:
#0C1017

Panel:
#11161F

Accent system

Recovery / positive

Emerald

Risk

Amber

Critical

Red

AI

Electric violet / indigo

Neutral data

Cool gray

Do not use color everywhere.

Color should communicate semantic state.

27. Typography

Use a premium modern sans-serif for UI.

Recommended:

Inter

or:

Geist

For numerical financial metrics, use tabular numerals.

Optional secondary typography:

JetBrains Mono

for:

- audit IDs,
 - webhook payloads,
 - timestamps,
 - event names,
 - policy decisions.
-

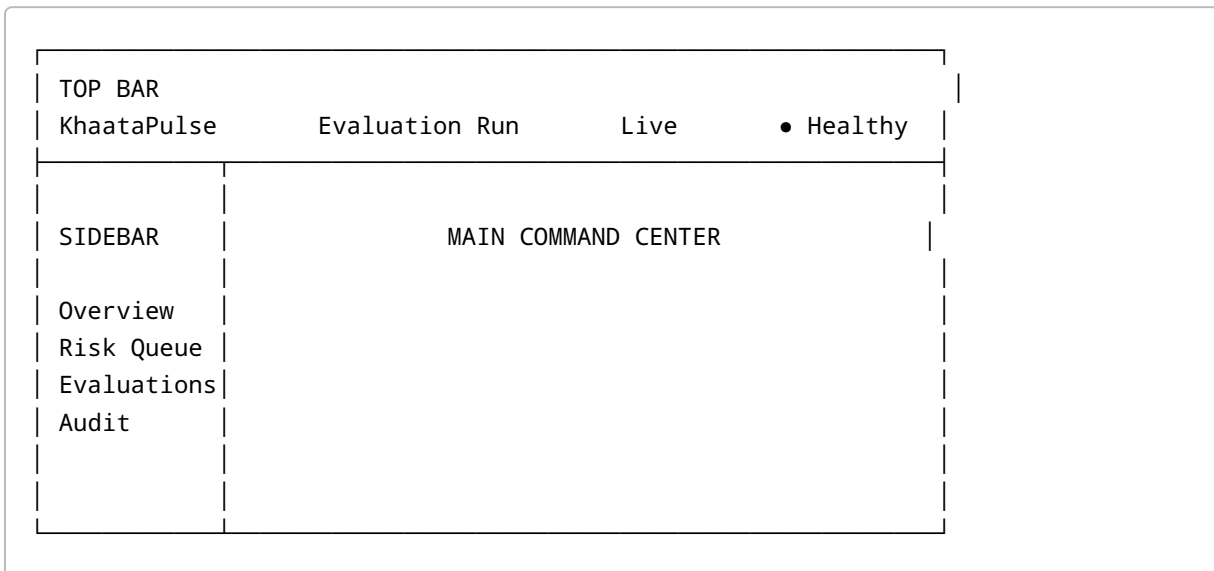
28. Frontend Layout

Desktop-first.

Target:

1440 × 900

Layout:



The sidebar should remain visually lightweight.

29. Hero Dashboard

The first viewport should immediately communicate:

KhaataPulse
Revenue Recovery Command Center

3,000 accounts evaluated

₹14.8L
Revenue Exposure

₹X
Recovered

+₹Y
Incremental Recovery

-Z%
Customer Contacts

The **incremental recovery number** should visually dominate.

30. Policy Comparison

Use a premium comparison matrix:

	STATIC	SMART	KHAATAPULSE
Recovered	₹X	₹Y	₹Z
Recovery Rate	X%	Y%	Z%
Contacts	X	Y	Z
Escalations	X	Y	Z

KhaataPulse should not simply be colored green.

Instead, visually emphasize the **delta**:

+₹40K
+2.8pp
-887 contacts
-47 escalations

31. Revenue Exposure Visualization

Use a horizontal flow:

₹14.8L Revenue Exposure



Healthy ₹10.0L	At Risk ₹3.2L	Recovered ₹1.6L
-------------------	------------------	--------------------

Clicking the at-risk segment opens the risk queue.

32. Risk Queue

The queue should feel like an operational control room.

REVENUE AT RISK		350 accounts		
CUSTOMER	RISK	AMOUNT	SIGNAL	ACTION
Arjun Mehta	78%	₹4,999	Billing	Grace
Kavita R.	91%	₹7,999	Fatigue	Block
Siddharth N.	88%	₹42,000	High value	Review

Risk should be communicated using a subtle heat scale rather than huge warning icons.

33. Revenue Time Machine

This is the signature interaction.

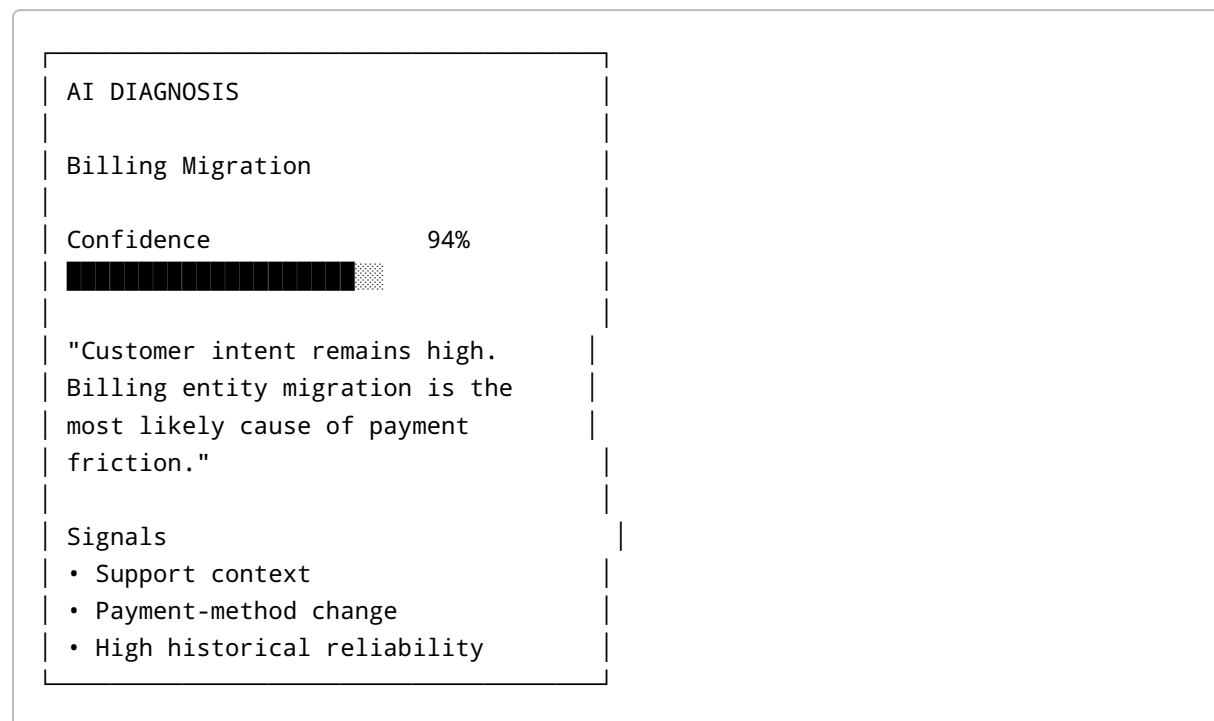
The screen should feel more cinematic than the dashboard.

ARJUN MEHTA ₹4,999 Renewal				
CUSTOMER STATE TIMELINE				
09:12	10:04	10:16	10:40	11:00
●	●	●	●	●
Invoice Viewed	Checkout Reopened	Payment Changed	Support Message	Risk Detected

Clicking an event expands contextual information.

34. AI Diagnosis Panel

Use a distinctive AI panel:



35. Intervention Decision View

Present actions as an economic decision surface.

INTERVENTION OPTIMIZER		
Action	Expected Net Revenue	
Smart Link	₹2,397	
Grace Period	₹3,241	← SELECTED
Silent Retry	₹1,882	
Human Call	₹2,610	

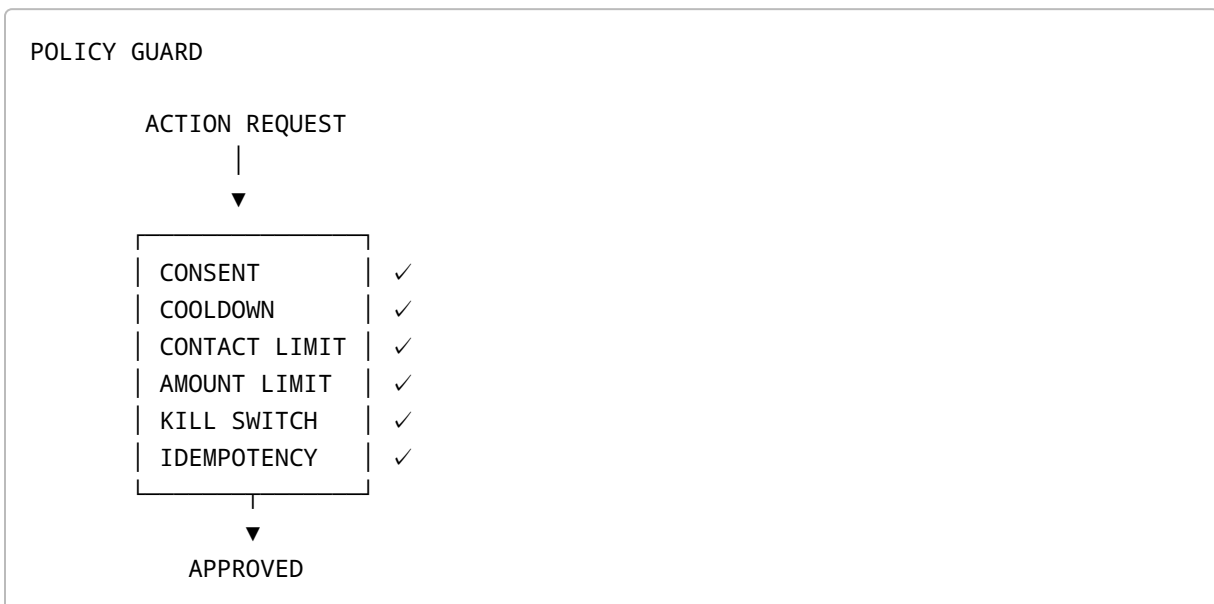
Then:

WHY GRACE PERIOD?

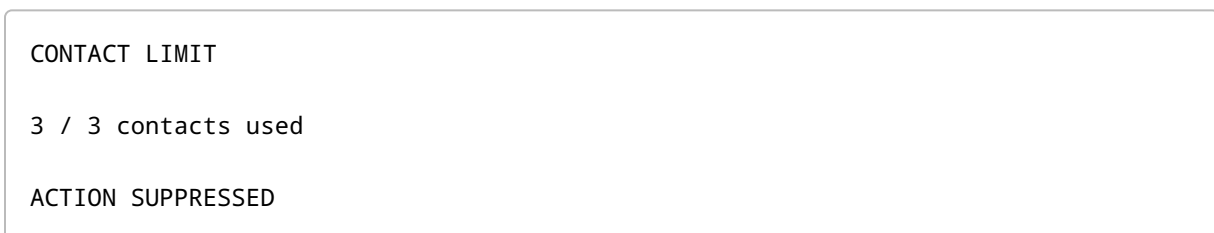
High payment intent
+
Billing transition
+
High churn sensitivity
=
Lower-pressure recovery

36. Policy Guard Visualization

The Policy Guard should feel like a security checkpoint.



For blocked actions:



This should be visually satisfying.

37. Audit Drawer

The audit drawer should resemble an enterprise event stream.

AUDIT TRAIL

```
11:00:02  risk_detected
          risk_score=0.78

11:00:03  diagnosis_generated
          cause=billing_migration

11:00:03  action_proposed
          action=grace_period

11:00:03  policy_check
          status=approved

11:00:04  action_executed
          gateway=payment_links

11:11:22  payment_received
          amount=₹4,999

11:11:23  case_closed
```

Each event should be expandable.

38. Gateway Payload Viewer

The audit drawer should have an expandable:

View Payload

```
{
  "event": "policy.action.executed",
  "idempotency_key": "rec_CASE_1042",
  "gateway_action": "POST /v1/payment_links",
  "amount": 499900,
  "currency": "INR"
}
```

Use monospace typography and syntax highlighting.

39. Live Webhook Interaction

The hero interaction:

```
[ SIMULATE PAYMENT WEBHOOK ]
```

Click.

Then show:

```
Webhook received
  ↓
Risk updated
  ↓
AI reasoning
  ↓
Action proposed
  ↓
Policy Guard
  ↓
Action executed
  ↓
Outcome
```

Use a real-time event animation.

The entire interaction should feel like a transaction moving through a financial nervous system.

40. Motion Design

Motion should communicate **system state**, not decoration.

Use:

Risk detected

A subtle amber pulse around the affected account.

AI reasoning

Small animated signal lines moving through the diagnosis panel.

Policy check

Sequential check indicators.

Approved

Green confirmation sweep.

Blocked

Short red interruption pulse.

Payment recovered

Metric counter animates upward.

Avoid:

- bouncing cards,
- excessive parallax,
- flashy particle effects,
- unnecessary page transitions.

41. Design System Components

Build reusable components:

```
MetricCard  
PolicyBadge  
RiskIndicator  
CustomerRow  
EventTimeline  
DiagnosisPanel  
InterventionTable  
PolicyGuard  
AuditDrawer  
PayloadViewer  
EvaluationMatrix  
RecoveryDelta  
StatusBadge
```

```
LiveEventStream  
SimulationButton
```

42. Design Tokens

Centralize:

```
colors  
spacing  
radii  
typography  
shadows  
motion  
z-index
```

Example:

```
--bg-primary  
--bg-secondary  
--surface  
--surface-elevated  
  
--text-primary  
--text-secondary  
--text-muted  
  
--accent-ai  
--accent-recovery  
--accent-warning  
--accent-critical  
  
--border-subtle  
--border-active
```

No arbitrary color values scattered across components.

43. Responsive Behavior

Desktop is the primary target.

Tablet:

Sidebar → collapsible
Comparison matrix → horizontally scrollable
Audit drawer → full-width drawer

Mobile:

Dashboard → stacked
Metrics → 2-column grid
Risk queue → cards
Time Machine → vertical timeline

The core desktop experience remains the priority.

44. Performance Contract

The application must feel instantaneous.

Dashboard

Precomputed evaluation:

< 500ms perceived load

Risk queue

Virtualize if necessary.

Live case

LLM response should be streamed or returned quickly.

Batch evaluation

Never block the UI.

Use:

```
POST /evaluation/run
  ↓
evaluation_run_id
  ↓
poll / subscribe
  ↓
results
```

The frontend must display progress.

45. Data Model

Minimum PostgreSQL entities:

```
customers
subscriptions
payments
events
recovery_cases
actions
audit_events
evaluation_runs
evaluation_results
```

46. Customer

```
Customer
-----
id
name
segment
ltv
subscription_id
created_at
```


47. Subscription

Subscription

id
customer_id
plan
amount
currency
renewal_at
status

48. Payment

Payment

id
customer_id
subscription_id
amount
status
failure_code
payment_method
created_at

49. Recovery Case

RecoveryCase

id
customer_id
risk_score
risk_level
diagnosis
diagnosis_confidence
proposed_action
selected_action
policy_status
outcome_status

```
created_at
closed_at
```

50. Audit Event

```
AuditEvent
-----
id
case_id
event_type
actor
payload
timestamp
idempotency_key
```

The `payload` should preserve structured information required for reconstruction.

51. Evaluation Run

```
EvaluationRun
-----
id
seed
cohort_size
simulator_version
model_version
policy_version
status
started_at
completed_at
```

52. Security and Privacy

Even though the MVP uses synthetic data:

- PII must be treated as sensitive.
- LLM prompts should contain only required fields.
- Production architecture should support PII redaction.

- Audit logs should avoid unnecessary personal information.
 - API keys must remain server-side.
 - No secret may be embedded in the frontend.
 - Payment credentials must never exist in the MVP database.
-

53. Idempotency

Every action must have an idempotency key.

Example:

```
rec_CASE_1042
```

Repeated execution of the same action must not produce duplicate recovery actions.

The action service must reject duplicate execution.

54. Kill Switch

A global merchant-level kill switch must exist.

AUTOMATION STATUS

- ACTIVE

Switching it off:

AUTOMATION STATUS

- PAUSED

must cause:

```
all new automated actions
```

↓

```
BLOCKED
```

Risk detection can continue.

No action may execute while the kill switch is active.

55. Observability

The backend should expose:

```
request latency
LLM latency
LLM failures
fallback count
policy blocks
action execution
evaluation duration
```

Basic structured logging is sufficient for MVP.

56. Environment Configuration

```
DATABASE_URL=
LLM_API_KEY=
LLM_MODEL=

APP_ENV=development

AUTO_ACTION_LIMIT=10000
MAX_CONTACTS_7D=3
CONTACT_COOLDOWN_HOURS=24

KILL_SWITCH=false
```

All policy thresholds should be configuration-driven, not scattered through code.

57. Local Development

Required:

```
docker compose up
```

Services:

```
frontend
backend
postgres
```

Optional:

```
llm provider
```

Demo mode must work even if the external LLM is unavailable.

58. Demo Mode

The application must have:

```
DEMO MODE
```

In demo mode:

- evaluation results are precomputed,
- hero customer is deterministic,
- LLM responses can be cached,
- gateway actions are simulated,
- audit events remain fully functional.

This guarantees the 3-minute demo cannot fail because of:

- network latency,
 - API rate limits,
 - LLM outage,
 - malformed response,
 - evaluation runtime.
-

59. Production Mode

Production architecture swaps:

```
Synthetic Event Generator
↓
Real Payment Webhooks
```

and:

```
Outcome Simulator
↓
Real Payment Outcomes
```

The agent, policy layer, audit layer, and frontend architecture remain largely unchanged.

60. Engineering Non-Goals

Do not optimize for:

- maximum number of agents,
- maximum LLM usage,
- maximum dashboard complexity,
- maximum model complexity,
- maximum number of integrations.

Optimize for:

```
Correctness
Explainability
Bounded execution
Evaluation integrity
Demo reliability
Visual quality
```

61. Definition of Done

The MVP is complete only when:

```
[ ] 3,000 customer cohort generated
[ ] Hidden simulator state isolated
[ ] Potential outcomes generated
[ ] Logistic Regression trained
```

- [] Risk sieve operational
- [] LLM diagnosis operational
- [] Pydantic validation operational
- [] LLM fallback operational
- [] Expected Net Revenue ranking operational
- [] Policy Guard operational
- [] Static Dunning implemented
- [] Smart Retry implemented
- [] KhaataPulse implemented
- [] Same-cohort evaluation implemented
- [] Incremental recovery calculated
- [] Multi-seed evaluation implemented
- [] Audit trail implemented
- [] Idempotency implemented
- [] Kill switch implemented
- [] Blocked case implemented
- [] Escalated case implemented
- [] Hero case implemented
- [] Dashboard implemented
- [] Revenue Time Machine implemented
- [] Audit drawer implemented
- [] Live webhook simulation implemented
- [] Demo mode implemented
- [] Docker deployment works

62. Final User Experience

The user should feel that they are operating a **financial intelligence control room**.

The visual hierarchy should be:

```
graph TD; MONEY --> RISK; RISK --> DECISION; DECISION --> CONTROL; CONTROL --> OUTCOME;
```

MONEY
↓
RISK
↓
DECISION
↓
CONTROL
↓
OUTCOME

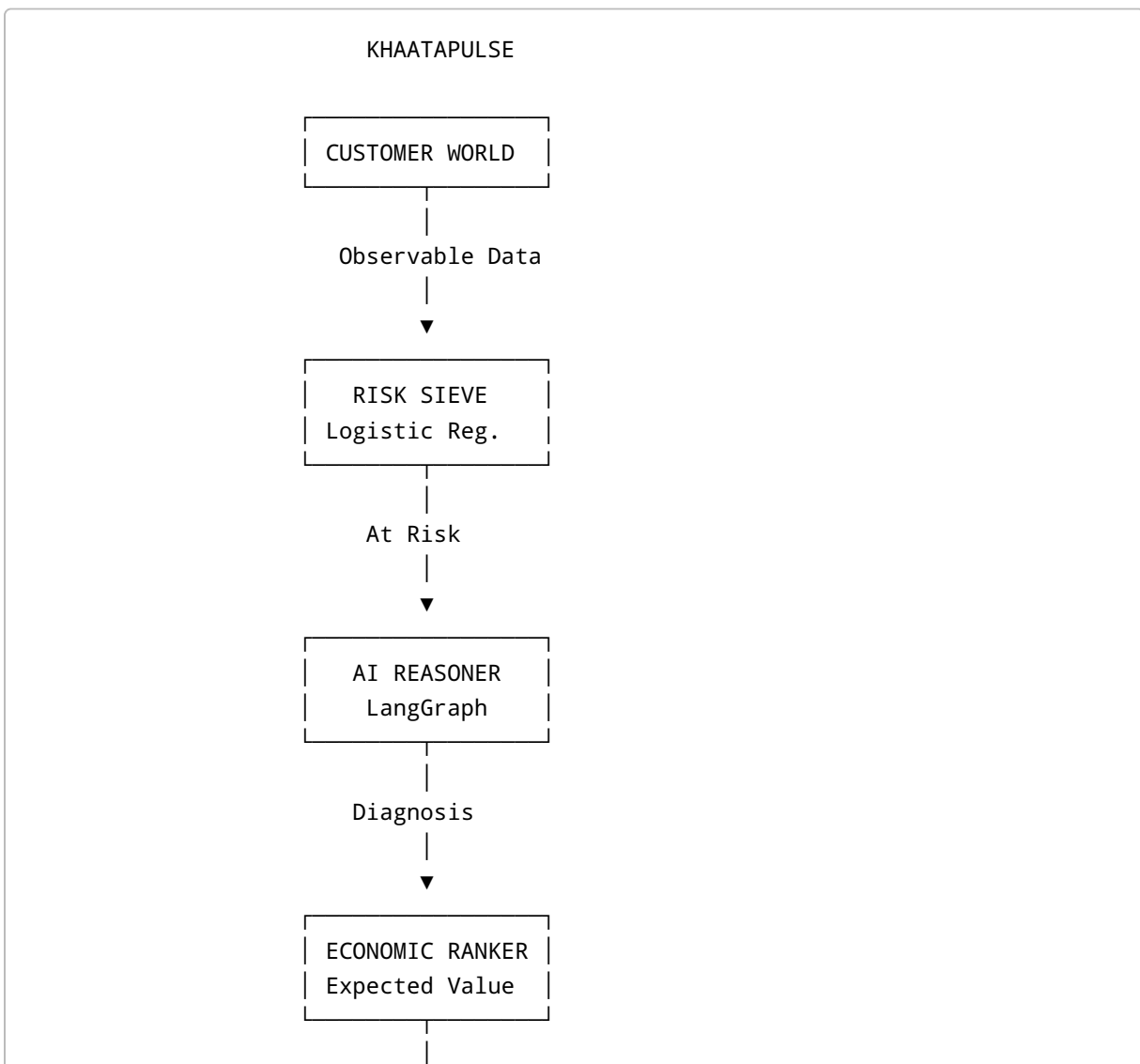
Not:

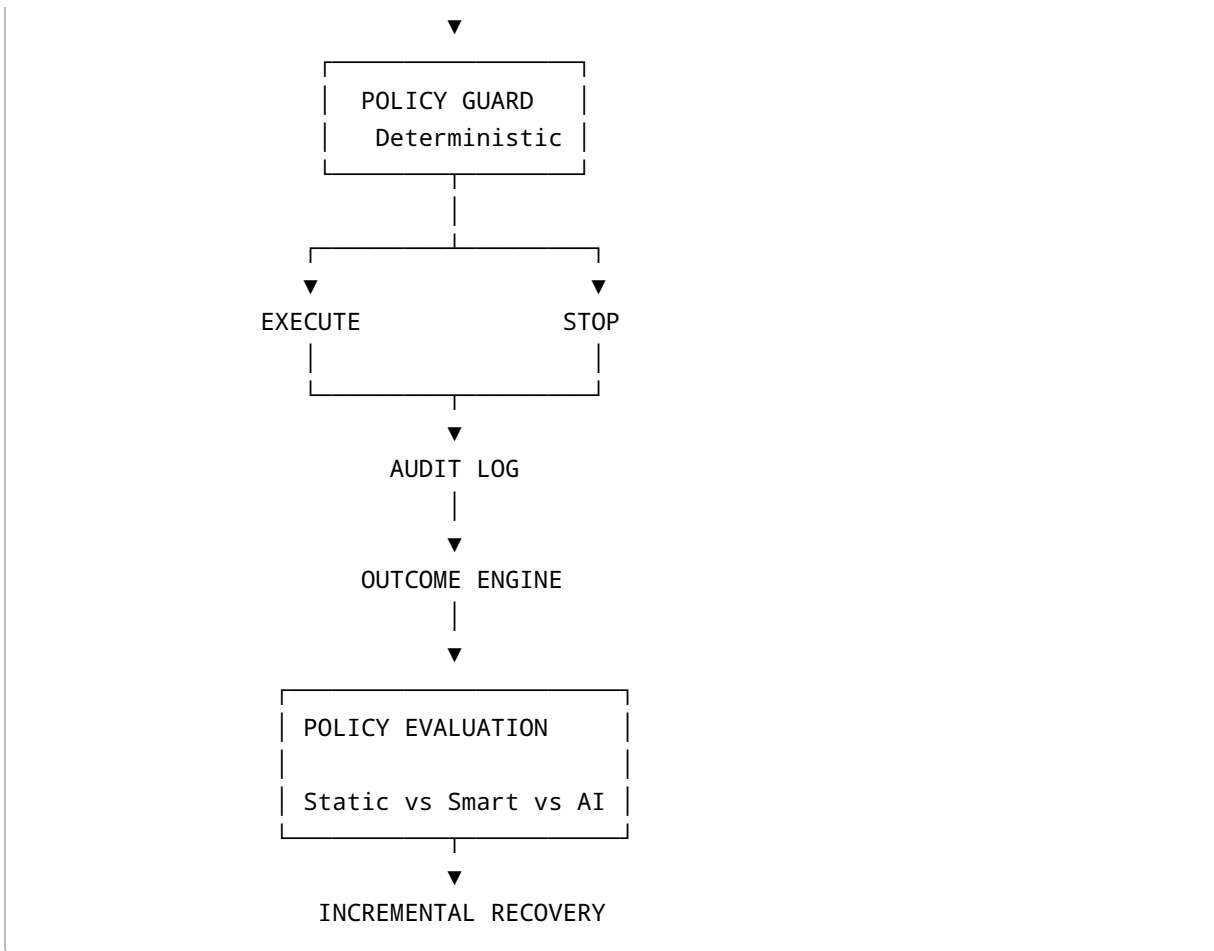
AI chatbot
↓
text response
↓
button

KhaataPulse should visually communicate that **money is moving through an intelligent but controlled system**.

63. Final Architecture Contract

The entire MVP can be reduced to this:





64. Final Product Statement

KhaataPulse is an AI Revenue Recovery Policy Engine that detects payment friction before failure, uses contextual AI to understand why a customer is at risk, optimizes for expected net revenue, enforces deterministic recovery boundaries, and evaluates its policy against realistic alternatives on the same customer cohort.

The engineering contract is intentionally strict:

The simulator knows the world. The agent only sees observations. The optimizer chooses economics. The Policy Guard controls authority. The audit trail records everything. The evaluation harness decides whether KhaataPulse actually wins.

And the frontend should make that architecture **feel visible** rather than merely displaying it:

A dark, premium financial command center where revenue exposure, AI reasoning, policy enforcement, and recovery outcomes move through one coherent visual system.